数据结构与算法分析-3

人工智能与自动化学院 谭毅华

yhtan@hust.edu.cn 87556301

上节课回顾

- 线性表
  - 可任意访问的线性结构
  - 顺序存储和链式存储实现基本操作
  - 插入和删除对两种存储方式时的实现有很大不同
  - 循环链表和双向链表可更灵活地访问数据
  - 以多项式加法为例说明线性表的应用
- 线性结构中的受限形式

栈

队列

3. 栈和队列

- 3.1栈
  - 栈的概念
  - 栈的存储结构和操作
  - 栈的应用举例
  - 栈与递归
- 3.2 队列
  - 队列的概念
  - 队列存储结构和操作
  - 队列的应用举例

前言

- 栈和队列是两种特殊的线性表，其特殊性在于栈和队列的基本操作是线性表的子集，它们是操作受限的线性表，可称为限定性的DS.

栈
队列
优先级队列

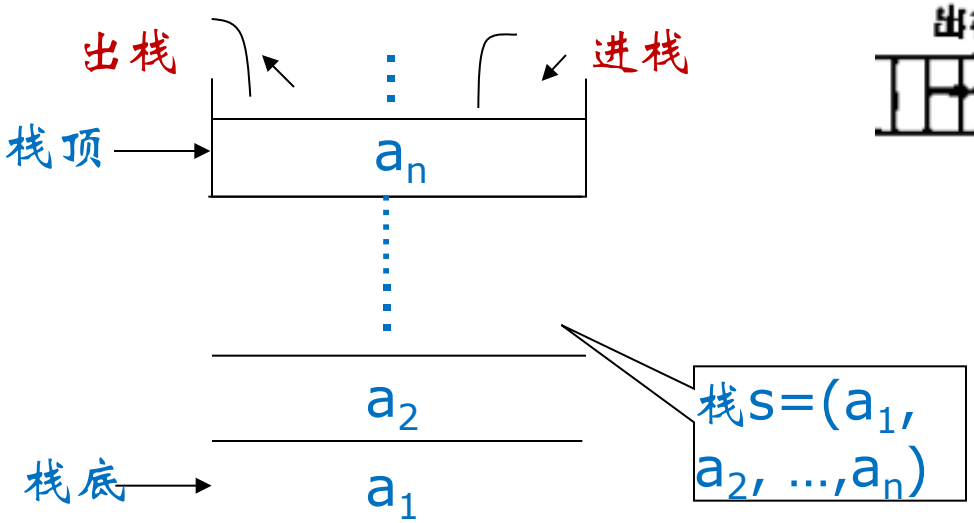
} 线性结构

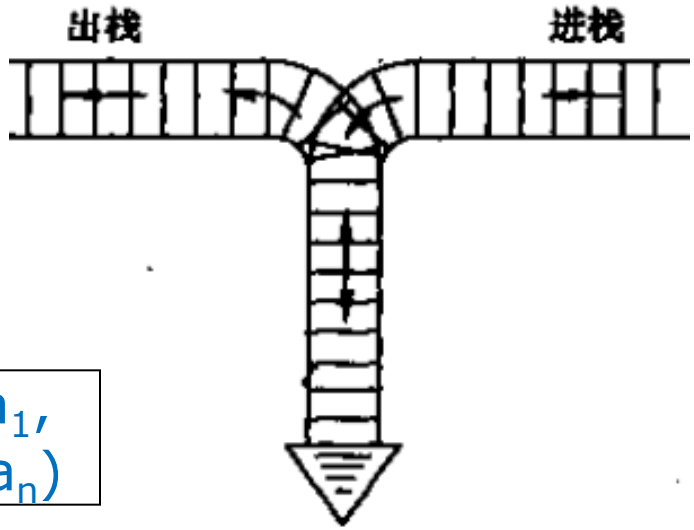
限定性：它们都是限制存取位置的线性结构

華中科技大學 人工智能与自动化学院

栈的概念

- 栈(stack)
  - 限定仅在表尾进行插入或删除操作的线性表
    - 表尾—栈顶
    - 表头—栈底
    - 不含元素的空表称空栈
  - 特点：先进后出（FILO）或后进先出（LIFO）

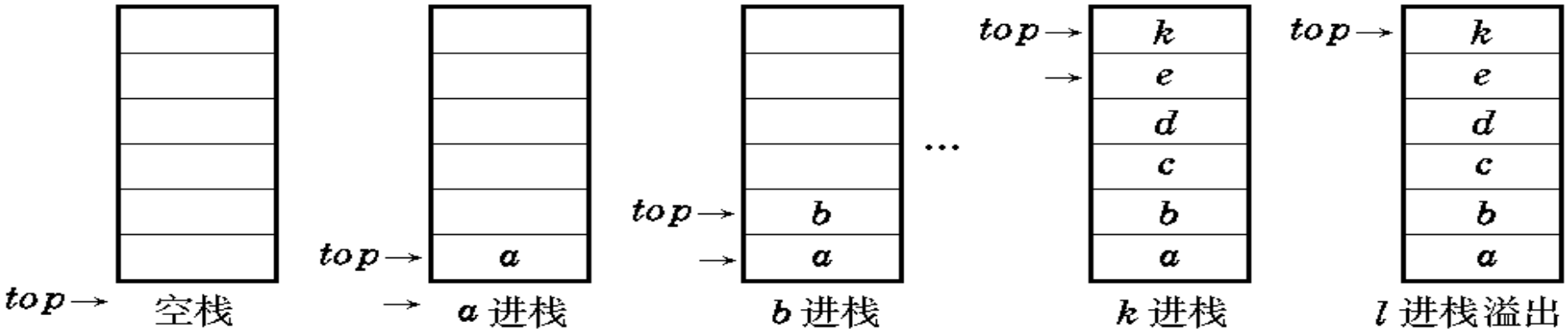




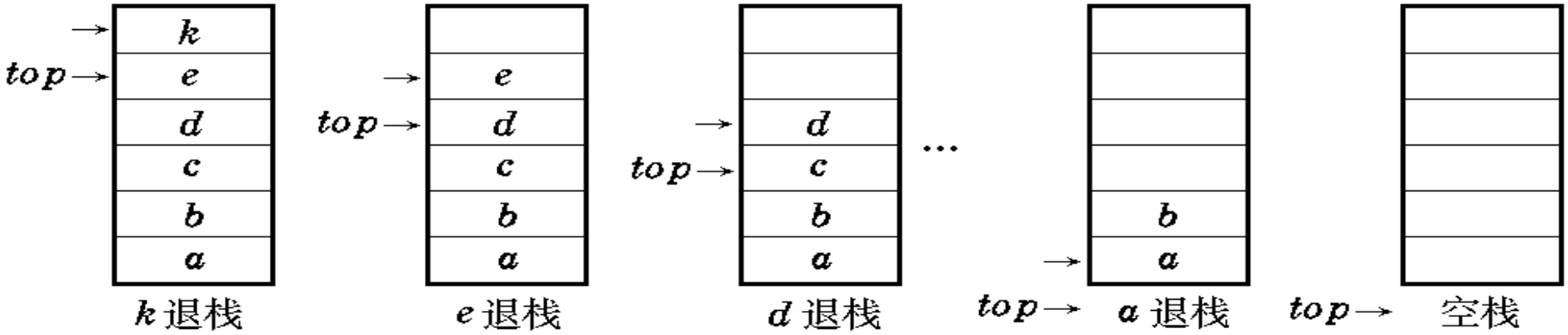
華中科技大学 人工智能与自动化学院

进出栈示例

- 进栈示例



- 出栈示例



華中科技大學 人工智能与自动化学院

栈的抽象数据类型

- 数据和关系

  $D=\{a_i|a_i \in ElemSet, i=1, 2, \ldots, n\geq 0\}$ //数据对象

  $R=\{<a_{i-1}, a_i>|a_{i-1},a_i \in D, i=1, 2, \ldots, n\}$ //数据关系。

  约定a_1为栈底，a_n为栈顶。

- 基本操作
  - 初始化
  - 进栈Push
  - 出栈Pop
  - 取栈顶元素GetTop
  - 判栈是否非空

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

栈的基本操作

InitStack(&S)

操作结果：构造一个空栈 S。

DestroyStack(&S)

初始条件：栈 S 已存在。

操作结果：栈 S 被销毁。

ClearStack(&S)

初始条件：栈 S 已存在。

操作结果：将 S 清为空栈。

StackEmpty(S)

初始条件：栈 S 已存在。

操作结果：若栈 S 为空栈，则返回 TRUE，否则 FALSE。

StackLength(S)

初始条件：栈 S 已存在。

操作结果：返回 S 的元素个数，即栈的长度。

GetTop(S, &e)

初始条件：栈 S 已存在且非空。

操作结果：用 e 返回 S 的栈顶元素。

Push(&S, e)

初始条件：栈 S 已存在。

操作结果：插入元素 e 为新的栈顶元素。

Pop(&S, &e)

初始条件：栈 S 已存在且非空。

操作结果：删除 S 的栈顶元素，并用 e 返回其值。

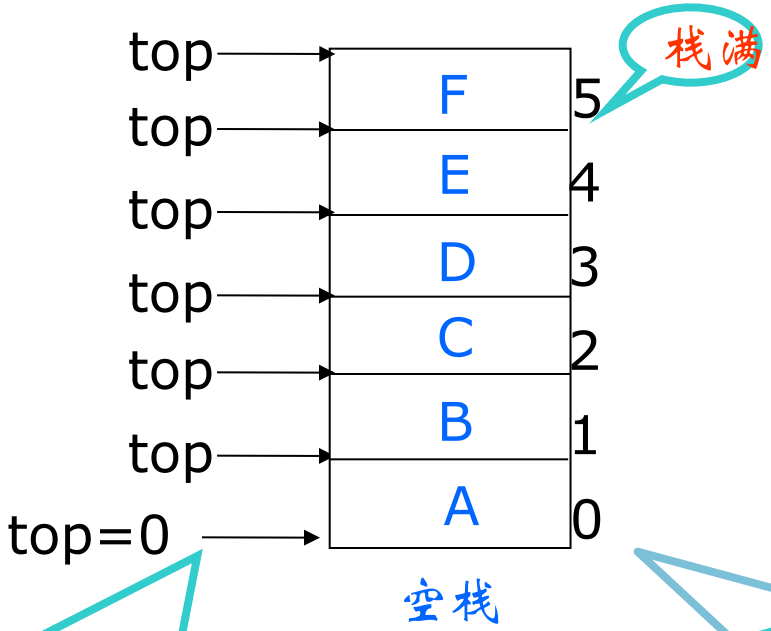
StackTraverse(S, visit())

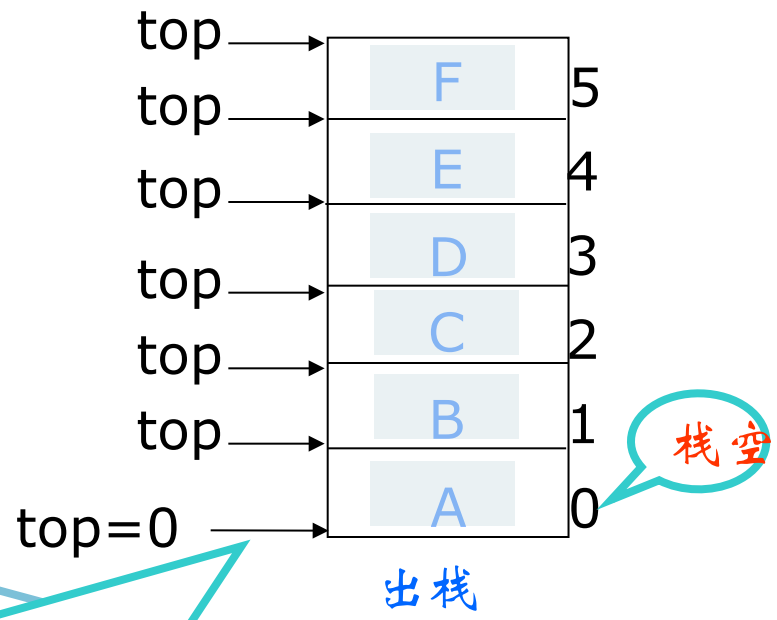
初始条件：栈 S 已存在且非空。

操作结果：从栈底到栈顶依次对 S 的每个数据元素调用函数 visit()。一旦 visit()失败，则操作失效。

栈的顺序存储结构

- 顺序栈—实现：一维数组s[M]
- 当堆栈中数据全部弹出后，在内存中的是什么？





栈顶指针top,指向实际栈顶后的空位置，初值为0

top=0栈空,此时出栈则下溢(underflow)
top=M栈满,此时入栈则上溢(overflow)

華中科技大學 人工智能与自动化学院

顺序栈的C语言实现

```
#define STACK_INIT_SIZE 100
#define STACKINCREMENT 10
Typedef struct {
   SElemType *base; //在栈构造之前和销毁之后，base值为NULL
   SElemType *top; //栈顶指针
   int stacksize;
} SqStack；
```


華中科技大學 人工智能与自动化学院

顺序栈的初始化操作

```
//初始化结构体的数据指针、栈顶、栈底以及栈的内存大小
Status InitStack(SqStack &S) {
    S.base = (SElemType*)malloc(STACK_INIT_SIZE*sizeof(SElemType));
    if(!S.base) exit(OVERFLOW);
    S.top = S.base;
    S.stacksize = STACK_INIT_SIZE;
    return OK;
}
```


華中科技大學 人工智能与自动化学院

顺序栈入栈操作

```
Status  push(SqStack &S, SElemType e) {
    //初始化条件
    if(S.top – S.base >= S.stacksize) {
        S.base = (SElemType *)realloc(S.base,
(S.stacksize+STACKINCREMENT)*siezeof(SElemType));
        if(!S.base) exit(OVERFLOW);
        S.top = S.base + S.stacksize;
    //更新栈的内存容量大小
        S.stacksize += STACKINCREMENT;
    }
    *S.top++ = e; //入栈
    return OK;
}
```


華中科技大學 人工智能与自动化学院

顺序栈栈顶相关操作

- 出栈算法

```
Status Pop(SqStack &S, SElemType &e) {
    if(S.top==S.base) return ERROR;
    e = *--S.top;
    return OK;
}
```

- 取栈顶算法

```
Status GetTop(SqStack S, SElemType &e) {
    if(S.top==S.base) return ERROR;
    e = *(S.top-1);
    return OK;
}
```

- 用数组实现的顺序栈的评价
  - 栈的容量在使用前难以估计
  - 操作简便

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

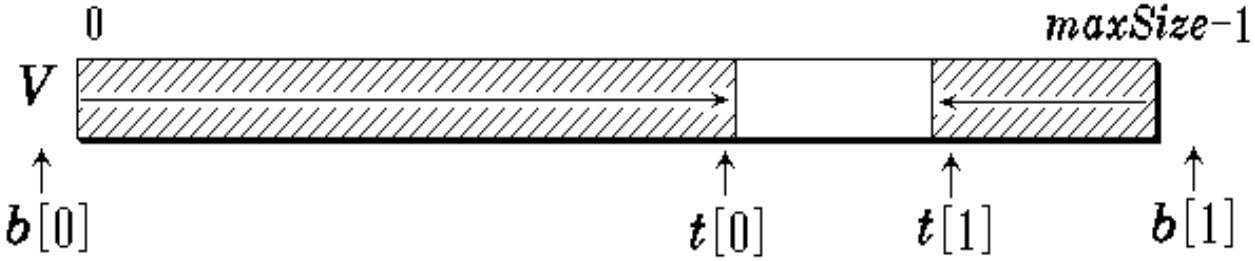
如何合理进行栈空间分配，以避免栈溢出或空间的浪费？

- 双栈共享一个栈空间（多栈共享栈空间）
- 栈的链接存储方式——链式栈

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

共享顺序栈

- 顺序栈所需容量在使用前难以估计，常在一个程序中将两个堆栈使用的空间放在一起。

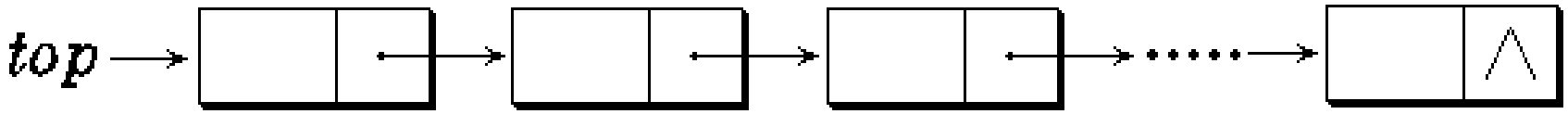


- 两个栈共享一个数组空间V[maxSize]
- 设立栈顶指针数组 t[2] 和栈底指针数组 b[2]
  t[i]和b[i]分别指示第 i 个栈的栈顶与栈底
- 初始 t[0] = b[0] = -1， t[1] = b[1] = maxSize
- 栈满条件：t[0]+1 == t[1]　　//栈顶指针相遇
- 栈空条件：t[0] = b[0] 或t[1] = b[1]
  //栈顶指针退到栈底

華中科技大學 人工智能与自动化学院

栈的链式存储结构

- 采用链表作为存储结构



  - 链式栈无栈满问题，空间可扩充
  - 插入与删除仅在栈顶处执行
  - 链式栈的栈顶在链头
  - 适合于多栈操作

- 节点定义

```
typedef  struct tagLinkedStack
{
    int data;
    struct tagLinkedStack *next;
} LinkedStack;
```

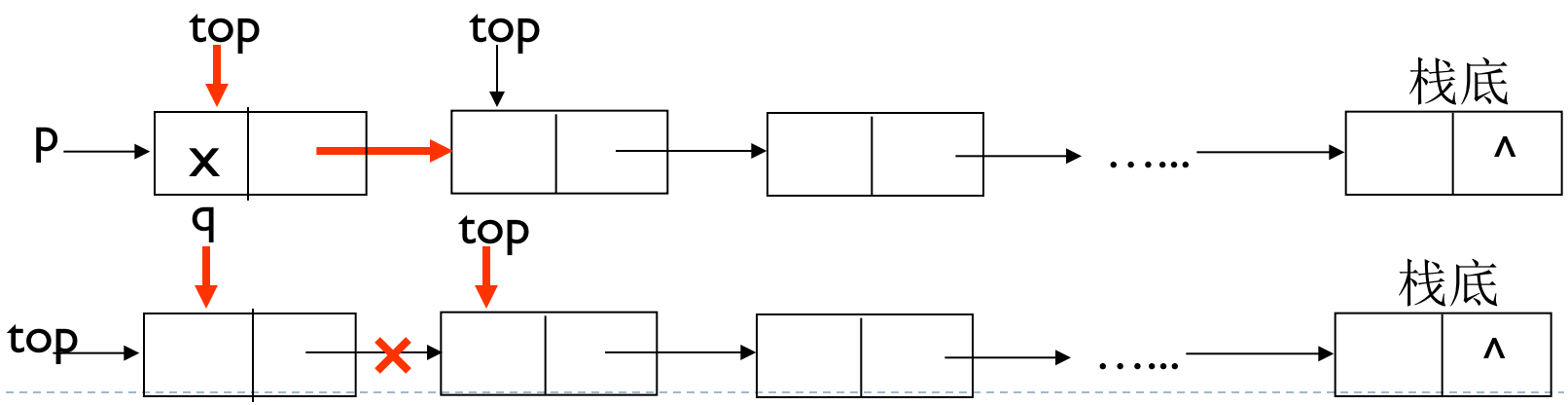

華中科技大學 人工智能与自动化学院

链栈的操作

- 链栈通常都在链表头端操作。

```
LinkedStack *LSPush(
   LinkedStack *top, int x)
{ LinkedStack *p;
   p =(LinkedStack *)
   malloc(sizeof(LinkedStack));
   p->data=x;
   p->next=top;
   top=p;
   return(p);
}
```

```
LinkedStack *LSPop(
    LinkedStack *top,int *x)
{   LinkedStack *q;
    if (top) {
        q=top;
        *x=top->data;
        top=top->next;
        free(q); }
    return(top);
}
```



華中科技大学 人工智能与自动化学院

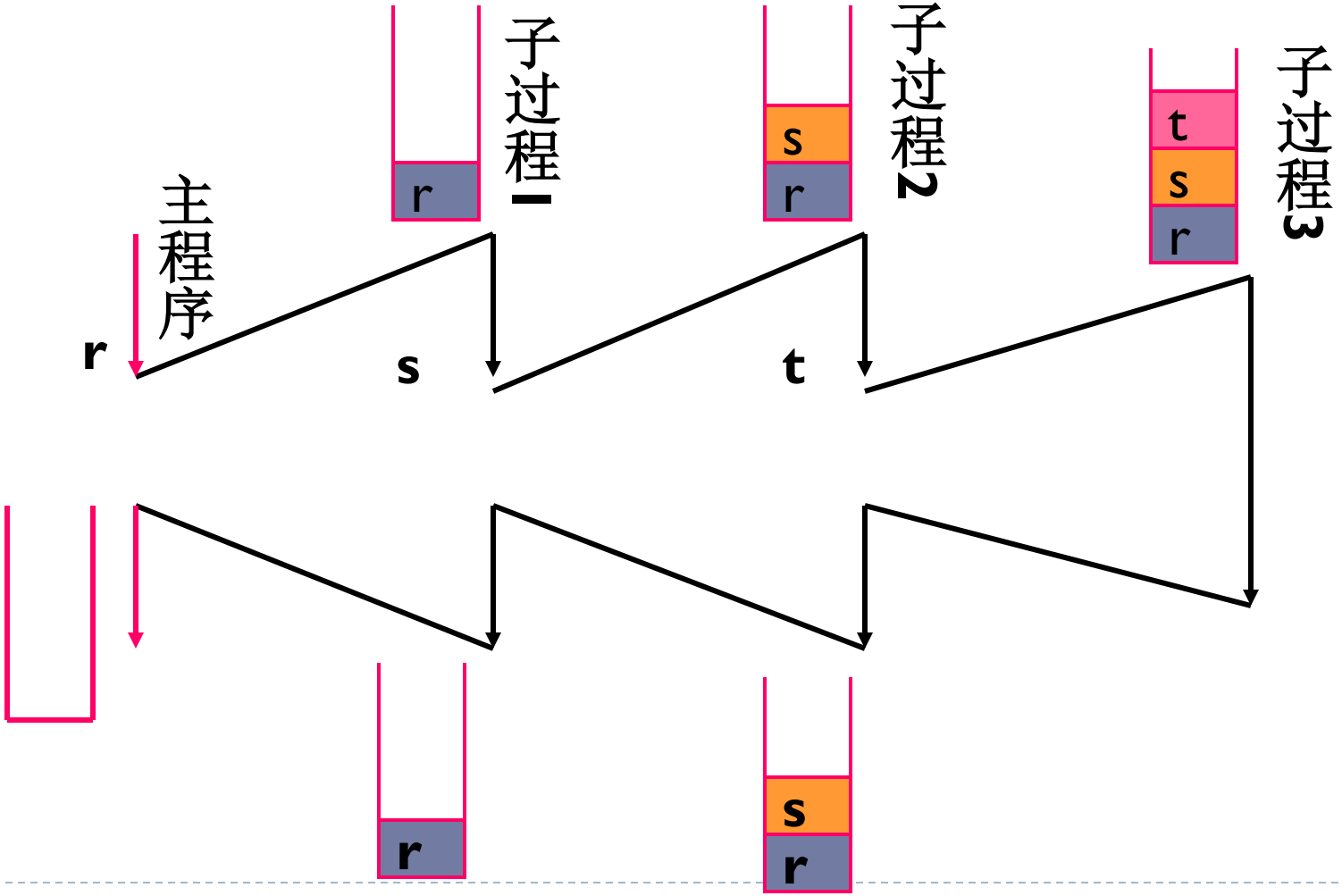
栈的应用

- 过程的嵌套调用
- 数制转换
- 走迷宫
- 表达式求值

華中科技大學 人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

过程的嵌套调用

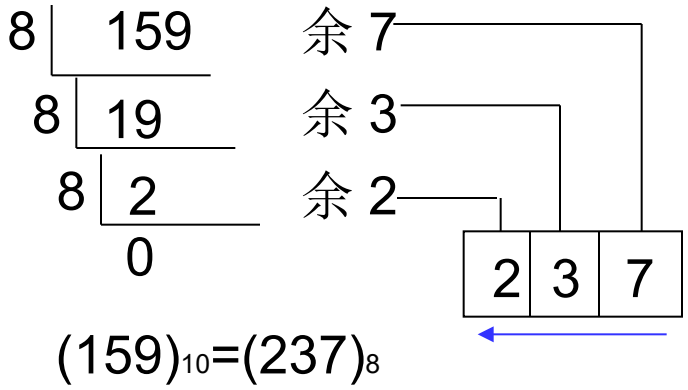
- 子程序调用-调用时入栈，返回时出栈

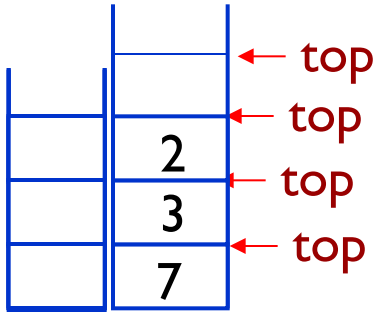


華中科技大學
人工智能与自动化学院

数制转换

- 把十进制数159转换成八进制数

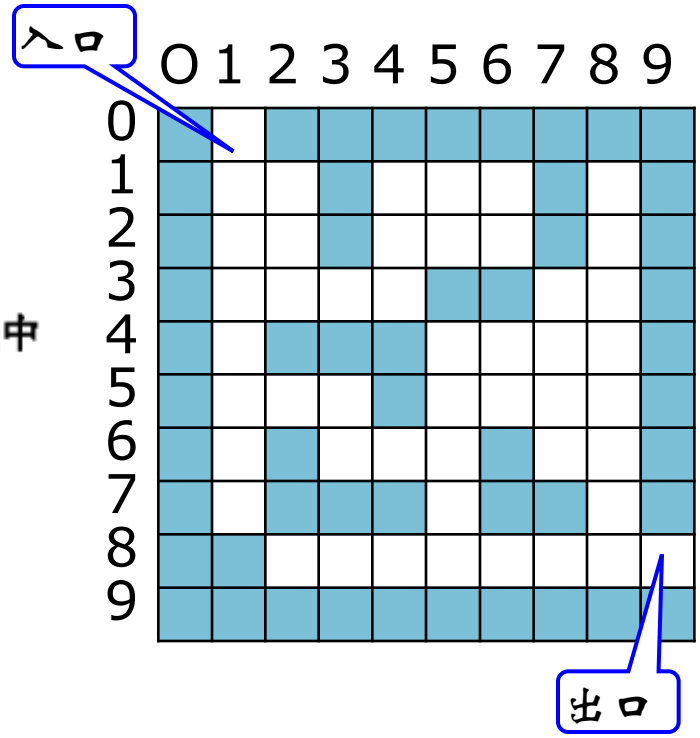





```
void conversion(){
    //构造空栈
    InitStack(s);
    scanf("%d",N);
    while(N){
        Push(S,N%8);
        N = N/8;
    }
    while(!StackEmpty){
        Pop(S,e);
        printf("%d",e);
    }
} //conversion
```

走迷宫

```
设定当前位置的初值为入口位置；
do {
  若当前位置可通，
  则{  将当前位置插入栈顶；          // 纳入路径
       若该位置是出口位置，则结束；// 求得路径存放在栈中
       否则切换当前位置的东邻方块为新的当前位置；
    }
  否则，
      若栈不空且栈顶位置尚有其他方向未经探索，
        则设定新的当前位置为沿顺时针方向旋转找到
              的栈顶位置的下一相邻块；
      若栈不空但栈顶位置的四周均不可通，
        则{  删去栈顶位置；// 从路径中删去该通道块
             若栈不空，则重新测试新的栈顶位置，
                 直至找到一个可通的相邻块或出栈至栈空；
          }
}while (栈不空);
```



華中科技大学
人工智能与自动化学院
A-ambitious 积极进取 U-unique 独意创新
T-tolerant 包容并蓄 O-open 开放交流

走迷宫-算法实现代码

```
Typedef struct{
 //通道块在路径上的序号
   int          ord;
//通道块在迷宫中的坐标位置
   PosType  seat;
//从此通道块走向下一通道块的方向
    int          di;
 }SElemType;
 //栈的元素类型
```

```
Status MazePath ( MazeType maze, PosType start, PosType end ) {
  // 若迷宫 maze 中存在从入口 start 到出口 end 的通道,则求得一条存放在栈中(从栈底到栈
  // 顶),并返回 TRUE;否则返回 FALSE
    InitStack(S);  curpos = start;          // 设定"当前位置"为"入口位置"
    curstep = 1;                            // 探索第一步
    do {
      if (Pass (curpos)) {  // 当前位置可以通过,即是未曾走到过的通道块
          FootPrint (curpos);               // 留下足迹
          e = ( curstep, curpos, 1 );
          Push (S,e);                       // 加入路径
          if (curpos == end)  return (TRUE);  // 到达终点(出口)
          curpos = NextPos ( curpos, 1 );   // 下一位置是当前位置的东邻
          curstep++;                        // 探索下一步

      }// if
      else {  // 当前位置不能通过
      if (!StackEmpty(S)) {
        Pop (S,e);
        while (e.di==4 && !StackEmpty(S)) {
          MarkPrint (e.seat);  Pop (S,e);       // 留下不能通过的标记,并退回一步
        }// while
        if (e.di<4) {
          e.di++;   Push ( S, e);               // 换下一个方向探索
          curpos = NextPos (e.seat e.di );      // 设定当前位置是该新方向上的相邻块
         }// if
       }// if
     }// else
    }while ( !StackEmpty(S) );
  return (FALSE);
}// MazePath
```


華中科技大學 人工智能与自动化学院 T-tolerant 包容并蓄 O-open 开放交流

表达式求值

- 运算规则：
  - 从左到右
  - 先乘除、后加减
  - 先括号内、后括号外
- 中缀表达式

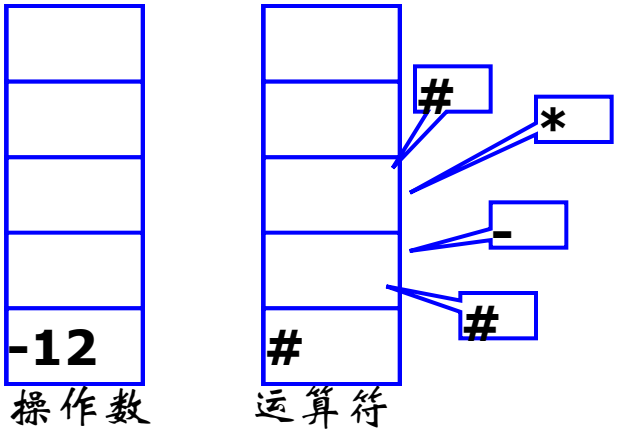
  a*b+c

  a+b*c

  a+(b*c+d)/e
- 操作数栈和运算符栈
- 例　计算 2+4-3*6
- 算法见教材P53.

算符间的优先关系

θ_1 \ θ_2	+	-	*	/	(	)	#
+	>	>	<	<	<	>	>
-	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=



華中科技大学 人工智能与自动化学院

表达式求值-算法实现代码

```
OperandType  EvaluateExpression() {
  // 算术表达式求值的算符优先算法。设 OPTR 和 OPND 分别为运算符栈和运算数栈，
  // OP 为运算符集合。
   InitStack (OPTR);   Push (OPTR,'#');
   initStack (OPND);   c = getchar();
   while (c!='#' || GetTop(OPTR)!='#') {
     if (!In(c, OP)){Push((OPND, c); c = getchar(); }     // 不是运算符则进栈
     else
       switch (Precede(GetTop(OPTR), c)) {
         case '<':   // 栈顶元素优先权低
               Push(OPTR, c);  c = getchar();
               break;
         case '=':   // 脱括号并接收下一字符
               Pop(OPTR, x);   c = getchar();
               break;

         case '>':   // 退栈并将运算结果入栈
              Pop(OPTR, theta);
              Pop(OPND, b);  Pop(OPND, a);
              Push(OPND, Operate(a, theta, b));
              break;
         }// switch
   }// while
   return GetTop(OPND);
}// EvaluateExpression
```

算法　3.4

问题1：为什么不将获得getchar()!='#'作为循环条件？
问题2：这样的循环条件下好处是什么？

问题1的答案：输入#结束后，如果仍然有多个运算符时没法进行循环处理
问题2的答案：如果上一轮输入的是优先级低的操作符仍然可以在本轮循环进栈，并且当输入的是#号时可以循环处理剩下的多个运算符

華中科技大學 人工智能与自动化学院

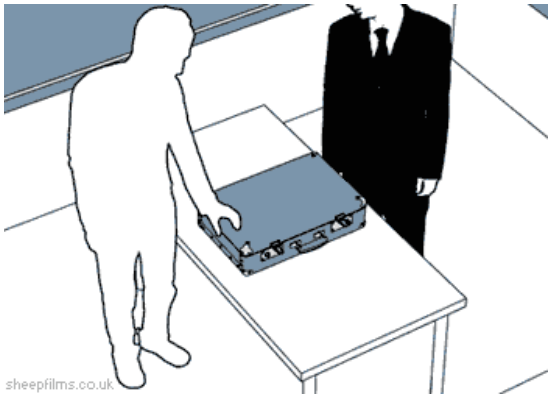
小结

- 栈是仅限顶端可访问数据的线性结构
- 先进后出、后进先出
- 关键的基本操作有GetTop,Push,Pop
- 其应用的关键是将问题对应的数据访问形成为栈结构处理

華中科技大学 人工智能与自动化学院

栈与递归

- 递归的定义
  - 若一个对象部分地包含它自己，或用它自己给自己定义，则称这个对象是递归的；若一个过程直接地或间接地调用自己，则称这个过程是递归的过程。
- 以下三种情况常常用到递归方法。
  - 定义是递归的
  - 数据结构是递归的
  - 问题的解法是递归的



情况一：定义是递归的

- 例，阶乘函数

$$n! = \begin{cases} 1, & \text{当} n = 0 \text{时} \\ n*(n-1)!, & \text{当} n \geq 1 \text{时} \end{cases}$$

求解阶乘函数的递归算法

```
long Factorial(long n) {
    if (n == 0) return 1;
    else return n*Factorial(n-1);
}
```


華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

阶乘计算过程

主程序 main : fact(4)

参数 4　计算 4*fact(3)　返回 24

参数 3　计算 3*fact(2)　返回 6

参数 2　计算 2*fact(1)　返回 2

参数 1　计算 1*fact(0)　返回 1

参数 0　直接定值 = 1　返回 1

递归调用

参数传递

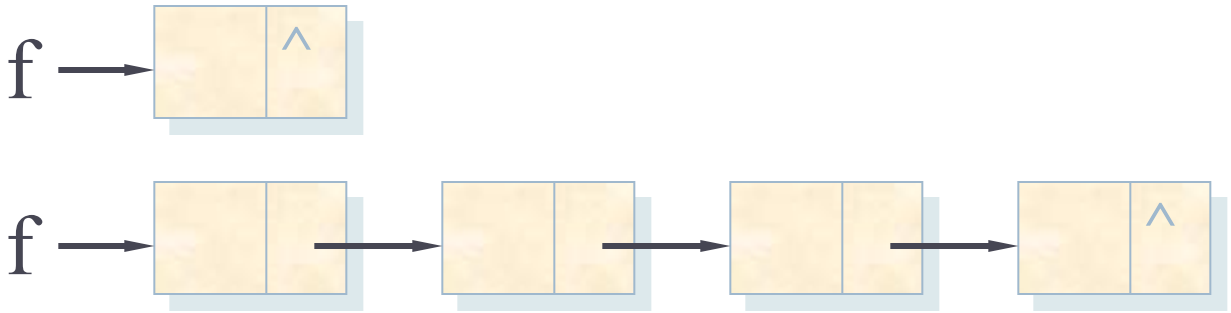
结果返回

回归求值

華中科技大學 人工智能与自动化学院

情况二：数据结构是递归的

- 例如，单链表结构



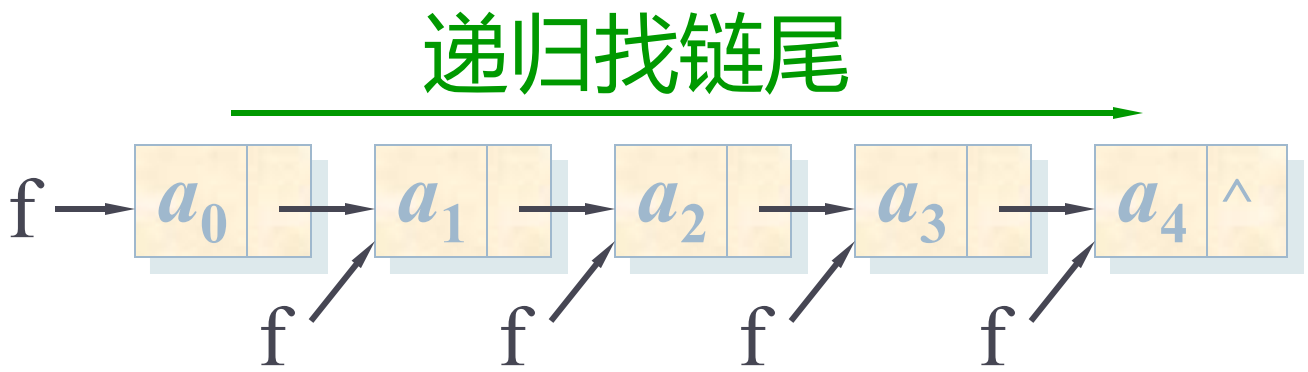
- 一个结点，它的指针域为NULL，是一个单链表；
- 一个结点，它的指针域指向单链表，仍是一个单链表。

華中科技大学 人工智能与自动化学院

例：

搜索链表最后一个结点并打印其数值

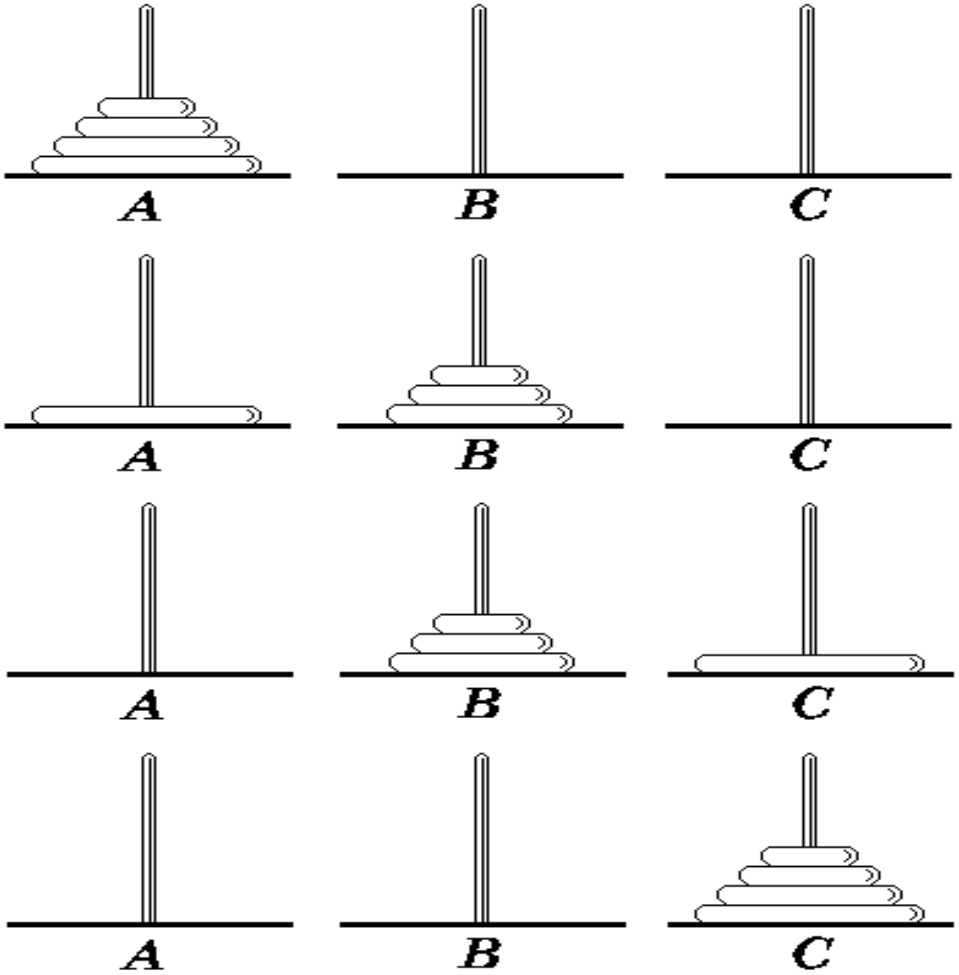
```
void Print(ListNode *f) {
    if (f->next == NULL)
        printf(f->data);
    else Print(f->next);
}
```



華中科技大學 人工智能与自动化学院

情况三：问题的解法是递归的

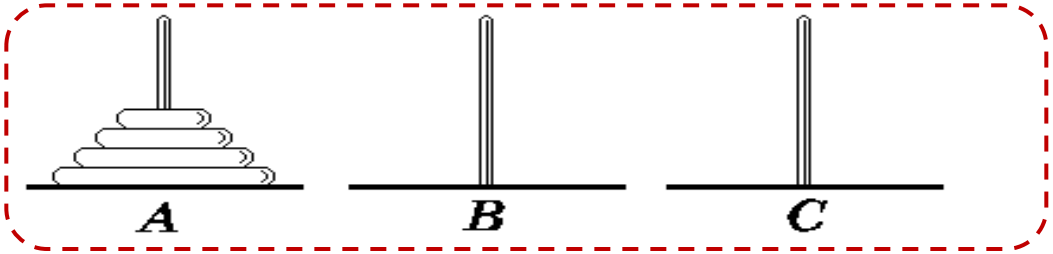
- 汉诺塔问题(Tower of Hanoi)



人工智能与自动化学院

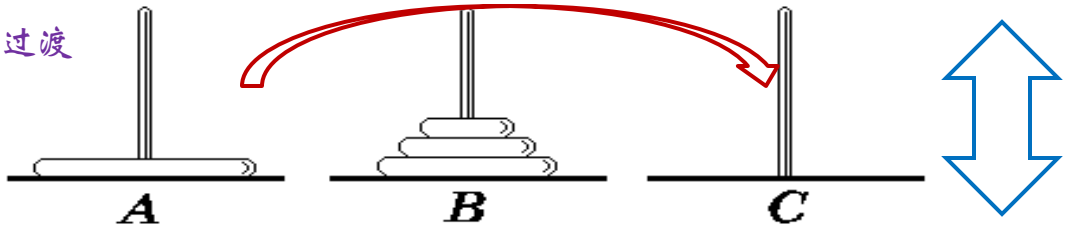
汉诺塔问题的解法：

初始状态

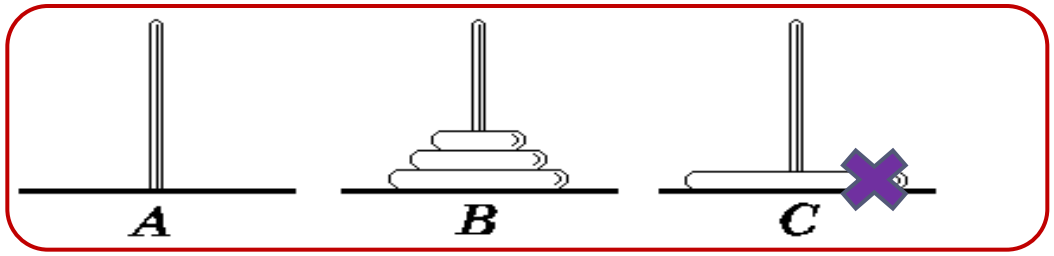


A至B，C为过渡

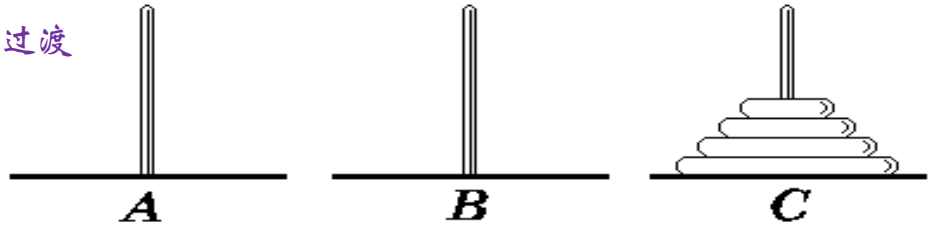
状态1



状态2



B至C，A为过渡



汉诺塔问题的解法：

如果 n = 1，则将这一个盘子直接从 A 柱移到 C 柱上。否则，执行以下三步：

① 用 C 柱做过渡，将 A 柱上的 (n-1) 个盘子移到 B 柱上；

② 将 A 柱上最后一个盘子直接移到 C 柱上；

③ 用 A 柱做过渡，将 B 柱上的 (n-1) 个盘子移到 C 柱上。

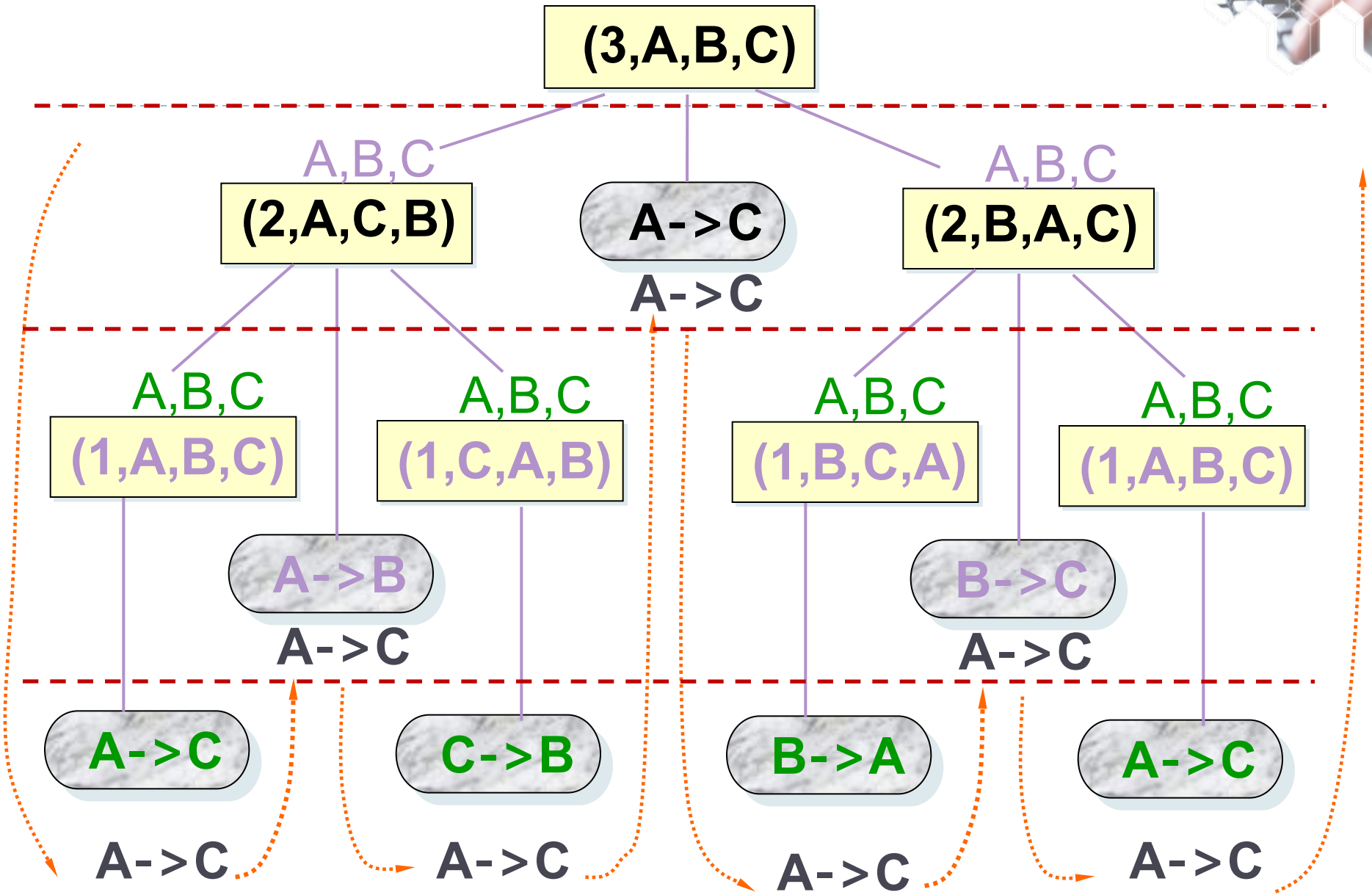
華中科技大學 人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

汉诺塔问题的代码实现

```
#include <iostream.h>
void Hanoi (int n, char A, char B, char C) {
//解决汉诺塔问题的算法
  //n是要挪的圆盘数，A是圆盘所在的初始柱子，B是过渡柱子，C是
  待//挪到的柱子
   if (n == 1) move(A,1,C);
   else {  Hanoi(n-1, A, C, B);   初始状态 ---→ 状态1
           move(A,n,C);           状态1 ---→ 状态2
           Hanoi(n-1, B, A, C);   状态2---→终态
        }
}
```

move函数中1,n代表第1和第n个圆盘

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流


(3,A,B,C)
A,B,C
(2,A,C,B)
A->C
A->C
A,B,C
(2,B,A,C)
A,B,C
(1,A,B,C)
A,B,C
(1,C,A,B)
A->B
A->C
A,B,C
(1,B,C,A)
A,B,C
(1,A,B,C)
B->C
A->C
A->C
A->C
C->B
A->C
B->A
A->C
A->C

构成递归的三个条件

- 不能无限制地调用本身，必须有一个出口，化简为非递归状况直接处理。

```
Procedure <name> ( <parameter list> ) {
  if ( < initial condition> )    //递归结束条件
    return ( initial value );
  else                           //递归
    return (<name> ( parameter exchange ));
}
```

递归法

递归法

递归法

从n到1

Hanoi 函数

```
if (n == 1) move(A,1,C);
```


華中科技大學 人工智能与自动化学院

构成递归的三个条件

- 2.递归过程在实现时，需要自己调用自己。

  层层向下递归，退出时的次序正好相反：

递归调用

$n! \Rightarrow (n-1)! \Rightarrow (n-2)! \Rightarrow \cdots \Rightarrow 1! \Rightarrow 0!=1$

返回次序

```
#include <iostream.h>
void Hanoi (int n, char A, char B, char C) {
//解决汉诺塔问题的算法
    if (n == 1) move(A,1,C);
    else {  Hanoi(n-1, A, C, B);
            move(A,n,C);
            Hanoi(n-1, B, A, C);
         }
}
```


華中科技大學 人工智能与自动化学院

构成递归的三个条件

- 3.递归过程可以转化为更为简单的相似递归过程。

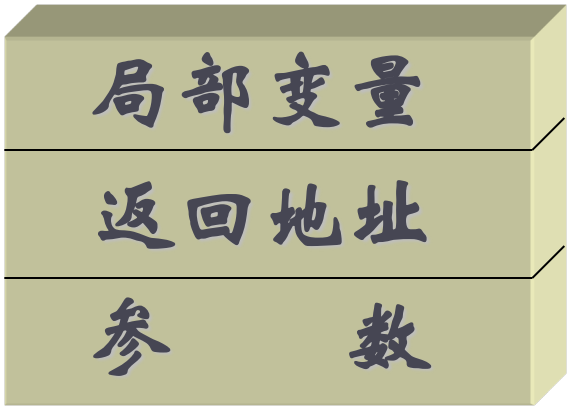
```
#include <iostream.h>
void Hanoi (int n, char A, char B, char C) {
//解决汉诺塔问题的算法
    if (n == 1) move(A,1,C);
    else {  Hanoi(n-1, A, C, B);
            move(A,n,C);
            Hanoi(n-1, B, A, C);
        }
}
```


華中科技大學 人工智能与自动化学院

递归工作栈

- 每一次递归调用时，需要为过程中使用的参数、局部变量等另外分配存储空间。
- 每层递归调用需分配的空间形成递归工作记录，按后进先出的栈组织。

活动记录框架



递归工作记录

華中科技大学 人工智能与自动化学院

主函数与过程函数示例

```
long Factorial(long n) {
   int temp;
   if (n == 0) return 1;
   else temp = n * Factorial(n-1);

   return temp;
}

void main() {
   int n;
   n = Factorial(4);

}
```

RetLoc2

RetLoc1

華中科技大學 人工智能与自动化学院

计算Fact时活动记录的内容

参数 返回值 返回地址 返回时的指令

递归调用序列

4	24	RetLoc1	return 4*6 //返回 24
3	6	RetLoc2	return 3*2 //返回 6
2	2	RetLoc2	return 2*1 //返回 2
1	1	RetLoc2	return 1*1 //返回 1
0	1	RetLoc2	return 1 //返回 1

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

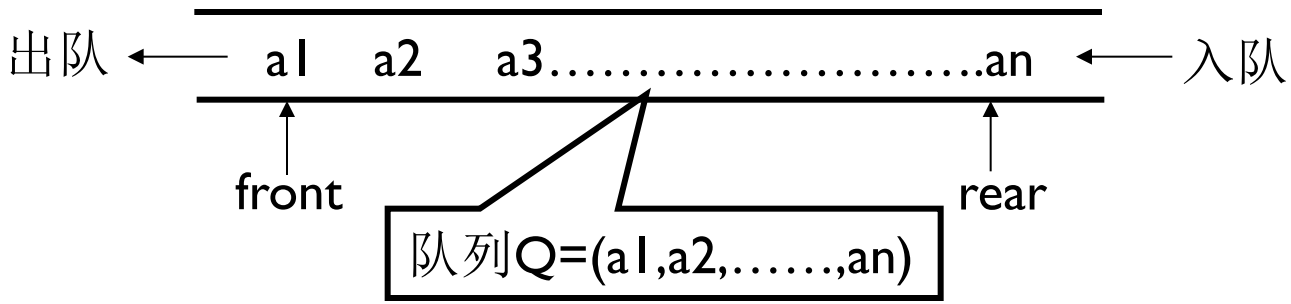
小结

- 递归是一个非常重要的思想：简化问题的解决
  - 递归必须有终止出口
  - 递归必须调用自身
  - 递归可以分解为相似的递归过程
- 递归隐含着栈的实现

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

3.2 队列

- 队列的定义及特点
  - 定义：队列是限定只能在表的一端进行插入，在表的另一端进行删除的线性表
    - 队尾(rear)——允许插入的一端
    - 队头(front)——允许删除的一端
  - 队列特点：先进先出(FIFO)



队列的抽象数据类型

ADT Queue {

数据对象：$D=\{a_i \mid a_i \in ElemSet, i=1,2,\cdots,n, n\geq 0\}$

数据关系：$R1=\{<a_{i-1},a_i> \mid a_{i-1}, a_i \in D, i=2,\cdots,n\}$

约定其中 a_1 端为队列头，a_n 端为队列尾。

基本操作：

InitQueue(&Q)

操作结果：构造一个空队列 Q。

DestroyQueue(&Q)

初始条件：队列 Q 已存在。

操作结果：队列 Q 被销毁，不再存在。

ClearQueue(&Q)

初始条件：队列 Q 已存在。

操作结果：将 Q 清为空队列。

QueueEmpty(Q)

初始条件：队列 Q 已存在。

操作结果：若 Q 为空队列，则返回 TRUE，否则 FALSE。

QueueLength(Q)

初始条件：队列 Q 已存在。

操作结果：返回 Q 的元素个数，即队列的长度。

GetHead(Q, &e)

初始条件：Q 为非空队列。

操作结果：用 e 返回 Q 的队头元素。

EnQueue(&Q, e)

初始条件：队列 Q 已存在。

操作结果：插入元素 e 为 Q 的新的队尾元素。

DeQueue(&Q, &e)

初始条件：Q 为非空队列。

操作结果：删除 Q 的队头元素，并用 e 返回其值。

QueueTraverse(Q, visit())

初始条件：Q 已存在且非空。

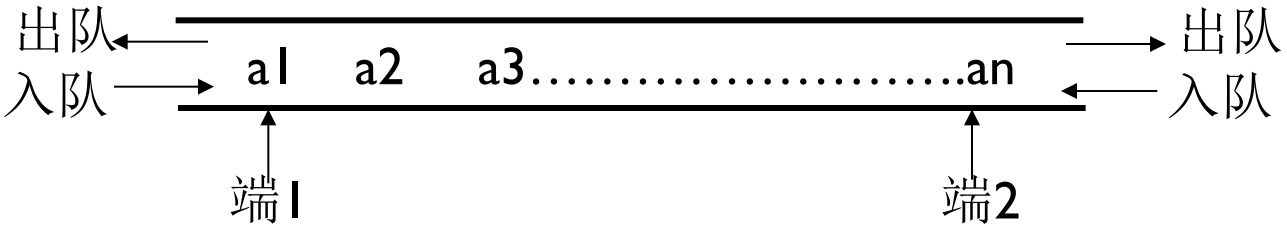
操作结果：从队头到队尾，依次对 Q 的每个数据元素调用函数 visit()。一旦 visit() 失败，则操作失败。

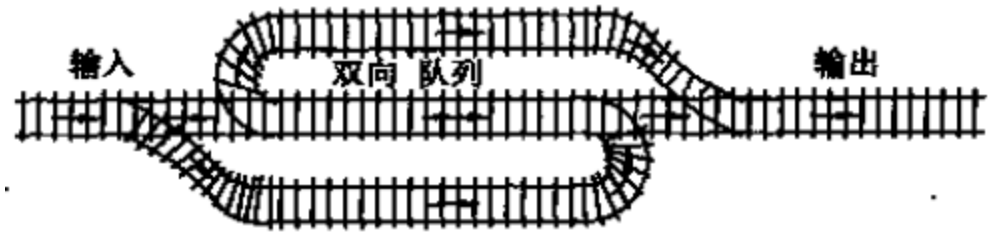
} ADT Queue

華中科技大學 人工智能与自动化学院

双端队列

- 定义：限定插入和删除操作可在表的两端进行的线性表。



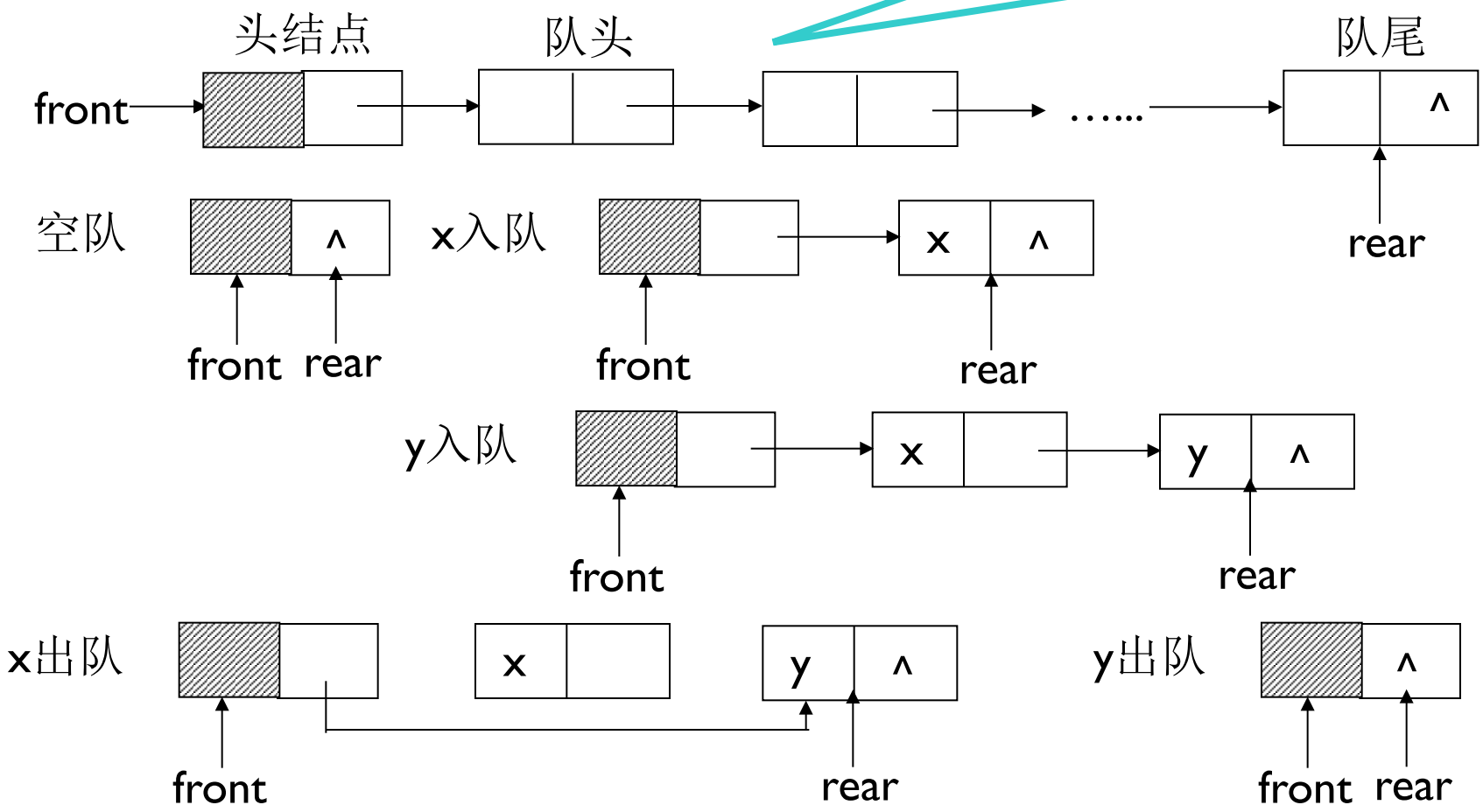


華中科技大學
人工智能与自动化学院

链队列的操作示意

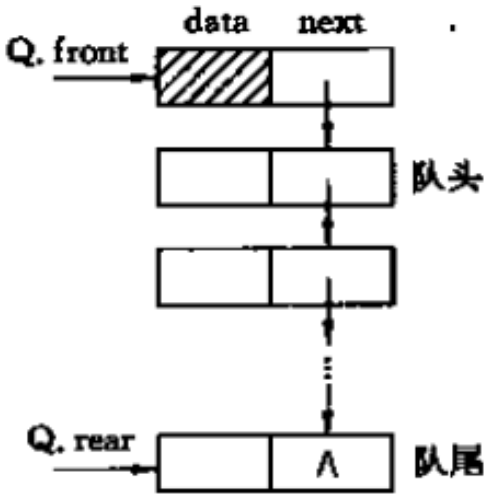
- 用链表表示的队列称链队列

设队首、队尾指针front和rear，front指向头结点，rear指向队尾



華中科技大學 人工智能与自动化学院

链队列基本操作




```
// 队列的链式存储结构
typedef struct QNode {
    QElemType        data;
    struct QNode     * next;
}QNode,  * QueuePtr;
typedef struct {
    QueuePtr  front; // 队头指针
    QueuePtr  rear;  // 队尾指针
}LinkQueue;
```

```
// 基本操作
Status InitQueue (LinkQueue &Q)
    // 构造一个空队列 Q
Status DestroyQueue (LinkQueue &Q)
    // 销毁队列 Q,Q 不再存在
Status ClearQueue (LinkQueue &Q)
    // 将 Q 清为空队列
Status QueueEmpty (LinkQueue Q)
    // 若队列 Q 为空队列,则返回 TRUE,
       否则返回 FALSE
int QueueLength (LinkQueue Q)
    // 返回 Q 的元素个数,即为队列的长度
Status GetHead (LinkQueue Q, QElemType &e)
    // 若队列不空,则用 e 返回 Q 的队头元素,
    // 并返回 OK;否则返回 ERROR
Status EnQueue (LinkQueue &Q, QElemType e)
    // 插入元素 e 为 Q 的新的队尾元素
Status DeQueue (LinkQueue &Q, QElemType &e)
    // 若队列不空,则删除 Q 的队头元素,用 e
    // 返回其值,并返回 OK;否则返回 ERROR
Status QueueTraverse(LinkQueue Q, visit())
    // 从队头到队尾依次对队列 Q 中每个元素
    // 调用函数 visit()。
    // 一旦 visit 失败,则操作失败。
```


華中科技大學 人工智能与自动化学院

链队列基本操作算法实现

```
// ----- 基本操作的算法描述(部分) -----
Status InitQueue (LinkQueue &Q) {
    // 构造一个空队列 Q
    Q.front = Q.rear = (QueuePtr)malloc(sizeof(QNode));
    if (!Q.front) exit (OVERFLOW);                    //存储分配失败
    Q.front->next = NULL;
    return OK;
}

Status DestroyQueue (LinkQueue &Q) {
    // 销毁队列 Q
    while (Q.front) {
      Q.rear = Q.front->next;
      free (Q.front);
      Q.front = Q.rear;
    }
    return OK;
}
```

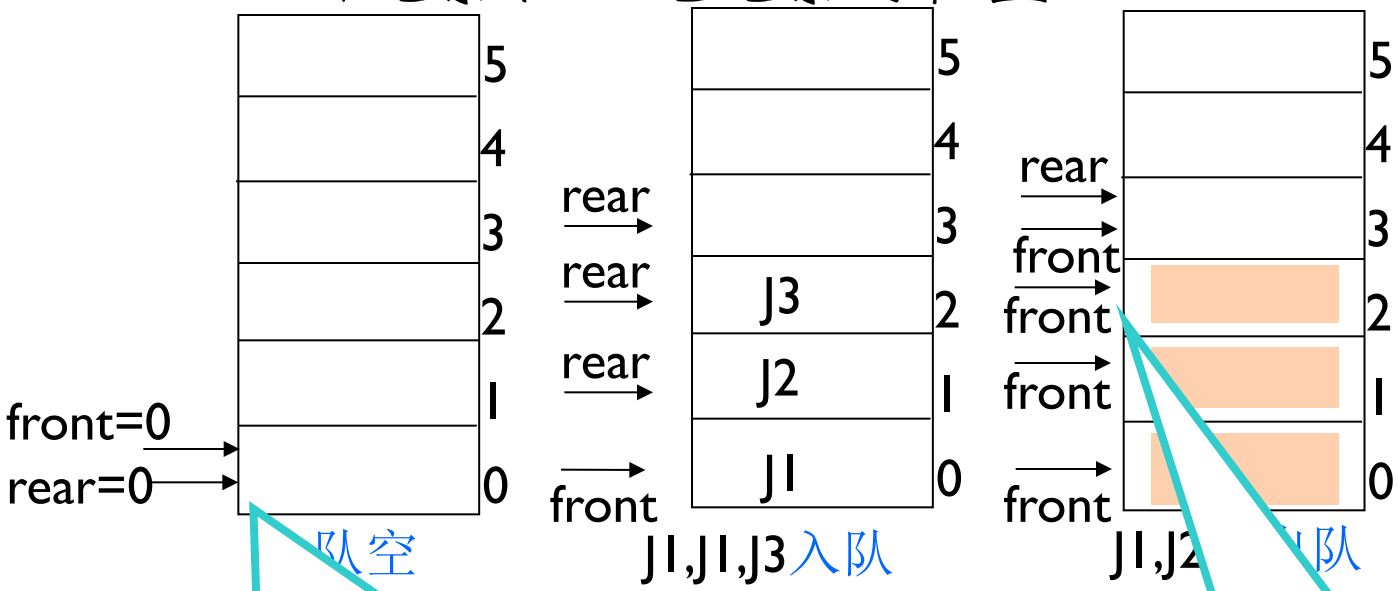
```
Status EnQueue (LinkQueue &Q, QElemType e) {
    // 插入元素 e 为 Q 的新的队尾元素
    p = (QueuePtr) malloc (sizeof (QNode));
    if (!p)  exit (OVERFLOW);                         //存储分配失败
    p->data = e;    p->next = NULL;
    Q.rear->next = p;
    Q.rear = p;
    return OK;
}

Status DeQueue (LinkQueue &Q, QElemType &e) {
    // 若队列不空,则删除 Q 的队头元素,用 e 返回其值,并返回 OK;
    // 否则返回 ERROR
    if (Q.front == Q.rear) return ERROR;
    p = Q.front->next;
    e = p->data;
    Q.front->next = p->next;
    if (Q.rear == p) Q.rear = Q.front;
    free (p);
    return OK;
}
```


華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

队列的顺序表示

- 队列作为线性表，也可以用顺序存储结构存储。除了可以用一组地址连续的存储单元依次存放从队头到队尾的元素之外，还需附设两个指针front和rear分别指向队头元素和队尾元素的位置。



设两个指针front,rear,约定：
rear指示队尾元素的下一个位置；
front指示队头元素
初值front=rear=0

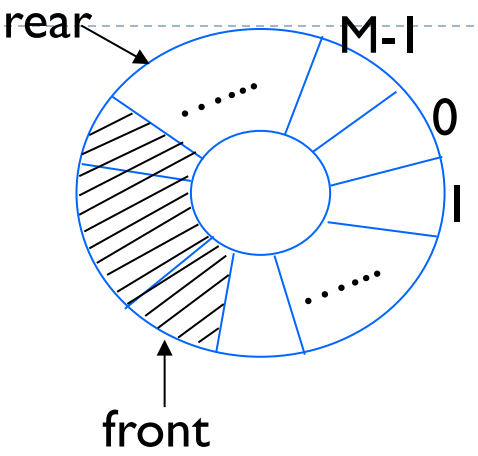
空队列条件：front==rear
入队列：sq[rear++]=x;
出队列：x=sq[front++];

顺序队列存在的问题

- 设数组维数为M，则-
  - 当front=0,rear=M时，当再次有元素入队时发生溢出——真溢出
  - 当front≠0,rear=M时，当再次有元素入队时发生溢出——假溢出
- 解决方案
  - 队首固定，每次出队剩余元素向下移动——浪费时间
  - 循环队列
    - 基本思想：把队列设想成环形，让sq[0]接在 sq[M-1]之后，若rear==M，则令rear=0;
    - 重复利用已经用过的空间

華中科技大學 人工智能与自动化学院

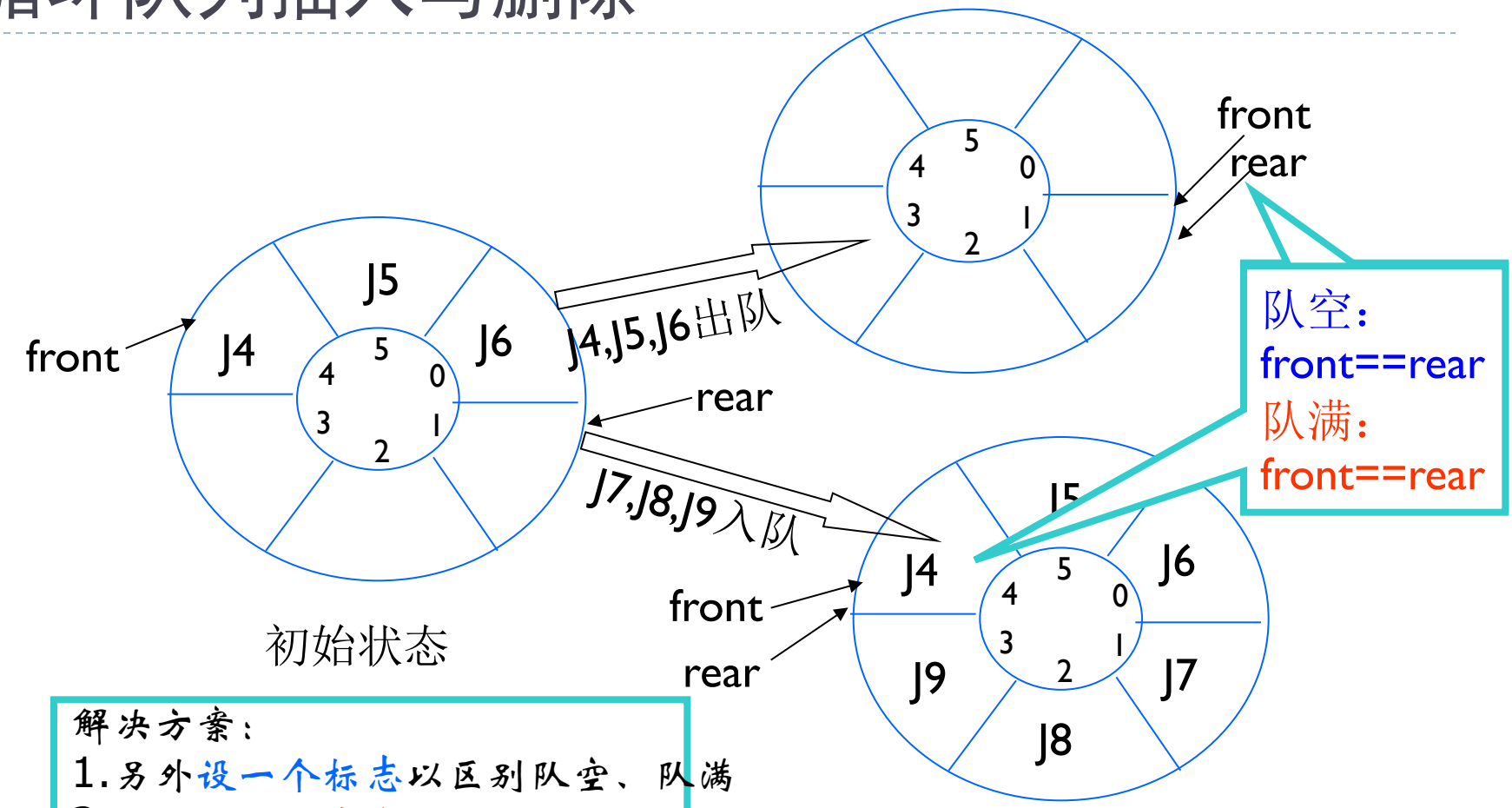
循环队列



- 基本思想：把队列sq设想成环形，让sq[0]接在sq[M-1]之后，若rear+1==M, 则令rear=0;
- 实现：利用“模”运算
  - 入队： sq[rear]=x; rear=(rear+1)%M;
  - 出队： x=sq[front];front=(front+1)%M;
- 队满、队空判定条件

華中科技大學 人工智能与自动化学院

循环队列插入与删除



初始状态

解决方案：

1. 另外设一个标志以区别队空、队满
2. 少用一个元素空间：

队空：front==rear

队满：(rear+1)%M==front

華中科技大學 人工智能与自动化学院

循环队列的顺序存储

```
#define MAXQSIZE 100
Typedef struct {
   QElemType *base;
   int front;
   int rear;
}SqQueue;
Status  InitQueue(SqQueue &Q) {
   Q.base = (QElemType *)malloc(MAXQSIZE*sizeof(QElemType));
   if(!Q.base) exit(OVERFLOW);
   Q.front = Q.rear = 0;
   return OK;
}
```

循环队列插入与删除

```
Status EnQueue(SqQueue &Q, QElemType e) {
    if((Q.rear+1)%MAXSIZE)== Q.front) return ERROR;
    Q.base[Q.rear] = e;
    Q.rear = (Q.rear+1)%MAXQSIZE;
    return OK;
} //入队
Status DeQueue(SqQueue &Q, QElemType &e) {
    if(Q.front == Q.rear) return ERROR;
    e = Q.base[Q.front];
    Q.front = (Q.front+1)%MAXQSIZE;
    return OK;
} //出队
```

小结

- 队列是另一种访问受限的线性结构
  - 队首和队尾可操作数据
  - 先进先出，后进后出
- 队列链式和顺序存储
- 顺序存储可使用循环队列解决上溢或下溢的问题

華中科技大學 人工智能与自动化学院

Take-home Message

- 栈
  - 顺序栈，共享栈，链式栈
  - 操作在表尾进行
- 栈与递归
  - 三种情况需要用到递归
  - 递归过程和递归工作栈
- 队列
  - 头删除，尾插入
  - 循环队列的实现